Assessment Overview

The objective of this assessment is to develop a console based application for mobile management using Java 8 features. Your task is to implement methods to add mobiles to the store,. The application should also filter and sort mobiles based on different criteria using lambda expressions, Java 8's Date & Time API.

Important Note

- Once you click Start Assessment, while we set up your lab environment, please review the Scenario and Objectives outlined in the Mission. Consider the logic you will use to implement the code to meet the assessment goals. If the lab isn't visible after 6 minutes, refresh your browser every 10 to 20 seconds until it loads.
- Read the objectives and test cases overview first to understand the assessment. Once you start the lab, you will see the project structure in the Project folder in the desktop.
- **Import the Project folder given in the desktop on the desired IDE you want to work upon.**
- The Project folder has the Assessment-Activities.pdf, which has the same activity details you are given below. You can go through it once the lab is started and work based on that.
- Open the project in IDE and develop the code.
- **Do not rename any classes, methods, or attributes provided in the stub code, and do not modify their visibility or access levels. Adhere strictly to the given structure.**
- **Your task is to write the required code only in the areas marked with 'TODO' comments. Do not change the Default Code Block sections.**
- If you have any step failed in your first attempt, you can take another attempt to correct / rework on only what was failed in your last attempt. On submit, the full assessment will be evaluated.
- Do not click Complete Assessment unless you are sure to end the assessment. Once you click Complete Assessment, you cannot take any more remaining attempts.
- You cannot copy paste anything into the lab. However, within the lab you can do it.

- Before submitting the assessment, make sure there are no breakpoints in the code and it is in a working condition to run the scripts.

What you will learn

By the end of this, you will be able to:

- Implement a Mobile Management System using Java 8 features.
- Utilize Lambda Expressions, Functional Interfaces, and the Date & Time API to filter, sort, and manage data.
- Apply modern Java features to build efficient and concise code.

What you will be provided

- Boilerplate code to develop the complete system.
- The boilerplate code is available in the lab Desktop > Project folder.
- **Import the Project folder given in the desktop on the desired IDE you want to work upon.**
- Instructions to guide you through the assessment tasks.
- A Java development environment pre-configured for you to start coding immediately.

What you need to know

- Proficiency in Java programming, particularly Java 8 features (Lambda Expressions, Functional Interfaces, Date & Time API).
- Understanding of how to work with collections and functional programming concepts in Java.
- How to work on Eclipse or IntelliJ to do the coding.

Scenario

The MobileStoreApp class serves as a console-based application to manage a list of mobile devices. It provides functionalities to add mobiles with details like brand, model, release date, and price, ensuring input validation. Users can filter, sort, and display mobiles based on criteria such as brand, release date, or both. It leverages Java 8 features like lambdas, streams, and predicates for efficient data processing

Here's how it works:

Add Mobiles: Allows users to add mobile details, including brand, model, release date, and price, with input validation.

Filter and Display: Provides options to filter mobiles by brand, release date, or a combination of both, and displays the results.

Sort Mobiles: Sorts and displays mobiles by their brand name.

Display by Date: Allows viewing mobiles released on or after a specific date.

Objectives

Options to be displayed as given below. Please use the exact inputs as given in the objectives for every option, in the same order and display the exact message after ever successful options selection**.**

addMobile() : It facilitates the addition of a new mobile device to a list of mobiles by capturing user input for key attributes such as brand, model, release date, and price. It ensures data integrity through validation.

filterByBrand() : It allows users to filter and view mobiles based on a specified brand using the Predicate<Mobile> to create a subset of mobiles that match the brand. If matching mobiles are found, it displays them; otherwise, it informs the user that no mobiles were found for the given brand.

displayMobilesReleasedAfterDate (): It allows users to filter and display mobiles released after a specified date using a Predicate<Mobile>. It ensures the entered date is in the correct format and prompts the user again if the input is invalid.If matching mobiles are found, it displays them; otherwise, it informs the user that no mobiles were released after the given date.

sortMobilesByBrand(): It sorts the list of mobiles alphabetically by their brand name using lambda expressions. This lambda expression defines how two Mobile objects (m1 and m2) should be compared based on their brand names.

filterMobilesByBrandAndReleaseDate(): It filters the list of mobiles based on two criteria: the brand and the release date and then creates a Predicate that checks

whether each mobile's brand matches the entered brand and whether the release date is after the specified date. It filters the list of mobiles using this combined predicate and displays the filtered list, or a message indicating no results were found if the list is empty.

displayMobilesByExactDate(): It filters and displays mobiles that were released on a specific date using Date- Time Api. It prompts the user to input a release date in the format "YYYY-MM-DD" and validates the input. Once a valid date is provided, the method filters the list of mobiles to find those whose release date exactly matches the entered date. It then displays the filtered list or a message if no mobiles are found for that date. Use

- **DateTimeFormatter** is used to define the pattern for parsing the date input in the format " YYYY -MM-DD".
- **LocalDate.parse()** is used to parse the input string into a LocalDate object,
- **DateTimeParseException**() to throw an exception , if the user enters date in the incorrect format.

displayMobiles(): It display a list of Mobile objects passed to it. If the list is empty, it outputs "No mobiles to display." Otherwise, it iterates over the list and prints each Mobile object using the toString() method.

A few Snapshots to give you an idea how the console application should be.

```
Menu
1. Add Mobile
2. Filter and Display Mobiles by Brand
3. Display Mobiles Released After a Specific Date
4. Sort and Display Mobiles by Brand
5. Filter and Display Mobiles by Brand and Release Date
6. Display Mobiles Released on a Specific Date
7. Exit
Enter your choice:
```

On entering invalid input:

```
Menu
1. Add Mobile
2. Filter and Display Mobiles by Brand
3. Display Mobiles Released After a Specific Date
4. Sort and Display Mobiles by Brand
5. Filter and Display Mobiles by Brand and Release Date
6. Display Mobiles Released on a Specific Date
7. Exit
Enter your choice: test
Invalid input. Please enter a valid integer.
```

On entering invalid brand name:

```
Menu
1. Add Mobile
2. Filter and Display Mobiles by Brand
3. Display Mobiles Released After a Specific Date
4. Sort and Display Mobiles by Brand
5. Filter and Display Mobiles by Brand and Release Date
6. Display Mobiles Released on a Specific Date
7. Exit
Enter your choice: 1
Enter Brand: 123
Invalid input! Brand name cannot contain numbers. Please enter a valid brand name.
```

On entering incorrect date format:

```
Menu
1. Add Mobile
2. Filter and Display Mobiles by Brand
3. Display Mobiles Released After a Specific Date
4. Sort and Display Mobiles by Brand
5. Filter and Display Mobiles by Brand and Release Date
6. Display Mobiles Released on a Specific Date
7. Exit
Enter your choice: 1
Enter Brand: realme
Enter Model: note
Enter the release date (yyyy-MM-dd): 2024/10/09
Invalid date format. Please enter a date in the format yyyy-MM-dd.
Enter the release date (yyyy-MM-dd):
```

On entering invalid price:

```
Menu
1. Add Mobile
2. Filter and Display Mobiles by Brand
3. Display Mobiles Released After a Specific Date
4. Sort and Display Mobiles by Brand
5. Filter and Display Mobiles by Brand and Release Date
6. Display Mobiles Released on a Specific Date
7. Exit
Enter your choice: 1
Enter Brand: realme
Enter Model: note
Enter the release date (yyyy-MM-dd): 2024-10-09
Enter Price: 1k
Invalid price format. Please enter a numeric value.
```

On successful mobile addition:

```
Menu
1. Add Mobile
2. Filter and Display Mobiles by Brand
3. Display Mobiles Released After a Specific Date
4. Sort and Display Mobiles by Brand
5. Filter and Display Mobiles by Brand and Release Date
6. Display Mobiles Released on a Specific Date
7. Exit
Enter your choice: 1
Enter Brand: redmi
Enter Model: note
Enter the release date (yyyy-MM-dd): 2024-10-09
Enter Price: 20000
Mobile added successfully
```

Filter and Display Mobiles by Brand:

```
Menu
1. Add Mobile
2. Filter and Display Mobiles by Brand
3. Display Mobiles Released After a Specific Date
4. Sort and Display Mobiles by Brand
5. Filter and Display Mobiles by Brand and Release Date
6. Display Mobiles Released on a Specific Date
7. Exit
Enter your choice: 2
Enter the brand to filter: redmi
ID = 5, Brand = redmi, Model = note, Release Date =2024-10-09, Price = 20000.0
```

Display Mobiles released after a specific date:

```
Menu
1. Add Mobile
2. Filter and Display Mobiles by Brand
3. Display Mobiles Released After a Specific Date
4. Sort and Display Mobiles by Brand
5. Filter and Display Mobiles by Brand and Release Date
6. Display Mobiles Released on a Specific Date
7. Exit
Enter your choice: 3
Enter the date (yyyy-MM-dd) to filter mobiles released after: 2024-10-08

Mobiles Released After 2024-10-08:
ID = 2, Brand = realme, Model = 123, Release Date =2024-10-09, Price = 20000.0
ID = 3, Brand = realme, Model = note, Release Date =2024-10-09, Price = 20000.0
ID = 4, Brand = reLME, Model = note, Release Date =2024-10-09, Price = 20000.0
ID = 5, Brand = redmi, Model = note, Release Date =2024-10-09, Price = 20000.0
```

Filter and Display Mobiles by Brand and Release Date:

```
Menu
1. Add Mobile
2. Filter and Display Mobiles by Brand
3. Display Mobiles Released After a Specific Date
4. Sort and Display Mobiles by Brand
5. Filter and Display Mobiles by Brand and Release Date
6. Display Mobiles Released on a Specific Date
7. Exit
Enter your choice: 5
Enter the brand to filter by: redmi
Enter the release date (yyyy-MM-dd) to filter mobiles released after: 2024-10-06
ID = 5, Brand = redmi, Model = note, Release Date =2024-10-09, Price = 20000.0
```

Final Notes

- Utilize the provided boilerplate code as your foundation.
- Focus on implementing the required features without altering the default code blocks.
- Good luck, and enjoy building your bonus calculating system!

Testcases

1. Check Build Status. (Marks: 10)
2. Check for Lambda Expressions in sortMobiles method. (Marks: 10)
3. Check for Date Time Api in displayMobilesByExactDate method . (Marks: 10)
4. Check Mobile Addition. (Marks: 10)
5. Check Filter and display mobiles by brand. (Marks: 10)
6. Check display mobiles released after a specific date. (Marks: 10)
7. Check Sort and display mobiles by brand. (Marks: 10)
8. Check Filter and display mobiles by brand and release date (Marks: 10)
9. Check Display mobiles released on a specific date (Marks: 10)
10. Check Successful Process Exit (Marks: 10)